

Tutorial - Semana 4 - Encontro 1

Introdução

Até a Semana 3, o Vertical Slice era um protótipo navegável: um `BP_Player` controlável por Enhanced Input, sobre um nível de teste com material, terreno, Nanite, Lumen e um primeiro build empacotado. A partir de agora a disciplina entra na Unidade II — Construir Sistemas — e a pergunta muda: como o jogo se estrutura por trás da jogabilidade visível? Este encontro abre essa unidade com dois papéis universais do Gameplay Framework — `GameMode` e `GameState` — que separam, respectivamente, a regra da partida do estado compartilhado dessa partida.

Este tutorial não substitui a explicação do professor em sala. Ele existe para que você possa acompanhar a implementação passo a passo durante o laboratório e revisar os passos depois da aula, sem depender da documentação oficial da Epic.

Objetivos da Semana

- Explicar `GameMode` e `GameState` como dois papéis universais e distintos do Gameplay Framework.
- Comparar a ausência de um equivalente direto a `GameMode/GameState` na Unity com o padrão de Managers/Singletons.
- Criar `BP_GameMode` e `BP_GameState` customizados e atribuí-los ao nível de teste do Vertical Slice.

Resultado Esperado ao Final da Semana

Um Vertical Slice operando sobre um Gameplay Framework próprio: `BP_GameMode` e `BP_GameState` customizados organizando regras e estado do nível de teste (Encontro 1), e `BP_PlayerController` e `BP_GameInstance` customizados fazendo a ponte com o jogador e garantindo persistência de um dado escolhido pelo grupo entre níveis (Encontro 2).

Pré-requisitos

- Projeto do Vertical Slice com o build da Semana 3 (material, Landscape, Nanite/Lumen ativos, primeiro build empacotado).
 - `BP_Player` funcional desde a Semana 2, controlável por Enhanced Input.
-

Antes de começar

O que você deverá possuir antes desta semana

- O projeto da Semana 3, aberto no Unreal Editor 5.6, com `Map_Exploration` consolidado e o `BP_Player` controlável sobre o terreno.

Arquivos necessários

- Nenhum arquivo externo é necessário neste encontro.

Assets utilizados

- Nenhum asset novo. Este encontro trabalha exclusivamente com classes de Blueprint do próprio projeto.

Projeto esperado

O mesmo projeto da Semana 3, pronto para receber as primeiras classes da pasta `Framework/` prevista no `PROJECT_ARCHITECTURE.md`.

Parte 1 — GameMode: quem decide as regras da partida

Objetivo

Compreender o GameMode como o papel universal responsável pela regra da partida, e criar um `BP_GameMode` customizado para o Vertical Slice.

Conceito

Toda engine de jogos — multiplayer-ready ou não — precisa de um lugar único que decida como uma partida começa, quando termina e quais são as condições de vitória ou derrota. Na Unreal, esse papel é formalizado por uma classe nativa: o **GameMode**. Ele existe apenas no servidor (mesmo em um projeto single-player como o Vertical Slice desta disciplina, a arquitetura da engine mantém essa separação) e concentra a lógica autoritativa de regras — nada relacionado à regra da partida deveria estar espalhado dentro do `BP_Player` ou de qualquer outro Actor do nível.

É importante entender que esse papel existe mesmo quando não há múltiplos jogadores conectados: a Unreal formaliza GameMode como arquitetura nativa desde a criação do projeto, independentemente do modo de jogo final.

Passo a passo

1. No Content Browser, abrir (ou criar) a subpasta `Framework/` dentro da estrutura de conteúdo do projeto, conforme a convenção do `PROJECT_ARCHITECTURE.md`.
2. Clicar com o botão direito dentro de `Framework/` e escolher Blueprint Class.
3. Na janela de seleção de classe pai, buscar por "Game Mode Base" e selecioná-la.
4. Nomear o novo Blueprint como `BP_GameMode`, conforme a convenção `BP_` do projeto.
5. Abrir o `BP_GameMode` recém-criado e, no painel Variables, adicionar uma variável simples de exemplo (por exemplo, um Boolean ou Integer) que represente uma regra de partida — não um estado a ser consultado por outros sistemas, mas uma decisão do próprio GameMode.
6. Compilar e salvar o Blueprint.

Resultado esperado

Um `BP_GameMode` criado dentro de `Framework/`, herdando de Game Mode Base, com uma variável de regra simples adicionada e compilada sem erros.

Verificando

Abrir novamente o `BP_GameMode` e confirmar, no painel Class Settings, que a classe pai exibida é "Game Mode Base" e que o Blueprint compila sem nenhum erro ou aviso vermelho no painel Compiler Results.

Problemas comuns

- **Escolher "Game Mode" em vez de "Game Mode Base":** ambas as opções aparecem na busca; para este projeto, "Game Mode Base" é suficiente e evita dependências desnecessárias de funcionalidades multiplayer não usadas na disciplina.
- **Criar o Blueprint fora da pasta `Framework/`:** revise a localização antes de prosseguir — a organização de pastas é parte da avaliação da semana.
- **Esquecer de compilar após adicionar a variável:** o Blueprint pode salvar mesmo sem compilar; sempre clicar em "Compile" antes de fechar.

Boas práticas

Nomeie a variável de regra de forma descritiva (por exemplo, `bPartidaAtiva` em vez de `bVar1`) — a clareza do nome já comunica que aquele dado é uma decisão de regra, e não um estado a ser lido por outros sistemas.

Comparação com Unity

A Unity não possui uma classe equivalente nativa a `GameMode`: o mesmo problema — centralizar a regra da partida — é resolvido por convenção do próprio time, tipicamente com um `GameManager` implementado como singleton (um `MonoBehaviour` com instância estática acessada globalmente). A diferença não é de capacidade, mas de formalização: a Unreal impõe um lugar único e nomeado para essa responsabilidade desde o primeiro projeto criado, enquanto na Unity essa organização depende inteiramente da disciplina da equipe de desenvolvimento.

Parte 2 — GameState: o que qualquer sistema pode consultar

Objetivo

Compreender o `GameState` como o papel universal responsável pelo estado compartilhado da partida, criar um `BP_GameState` customizado e atribuir `GameMode/GameState` ao nível de teste.

Conceito

Se o GameMode decide as regras, falta um lugar que qualquer sistema do jogo — a interface, um NPC, outro jogador — possa consultar para saber o estado atual da partida, sem precisar acessar diretamente a lógica autoritativa do GameMode. Esse é o papel do **GameState**: ele replica, para todos os clientes, um retrato do estado compartilhado da partida. A distinção prática entre os dois é um critério simples: se o dado precisa ser lido por outros sistemas, ele pertence ao GameState; se é uma decisão de como a partida se desenrola, pertence ao GameMode.

Passo a passo

1. Dentro de `Framework/`, criar um novo Blueprint Class herdando de "Game State Base".
2. Nomear o Blueprint como `BP_GameState`.
3. Abrir o `BP_GameState` e adicionar uma variável simples de exemplo que represente um estado a ser consultado por outros sistemas (por exemplo, uma pontuação ou um contador visível para a UI futura).
4. Compilar e salvar.
5. Abrir o `BP_GameMode` criado na Parte 1 e, no painel Class Defaults, localizar o campo "Game State Class" e atribuir `BP_GameState`.
6. Ir em Edit > Project Settings > Maps & Modes.
7. No campo "Default GameMode", selecionar `BP_GameMode`.
8. Confirmar que "Editor Startup Map" e "Game Default Map" continuam apontando para o nível de teste do projeto (`Map_Exploration` ou equivalente).
9. Testar em modo Play e, usando o console do Editor (Window > Developer Tools > Output Log) ou um Print String temporário, confirmar que o GameMode e o GameState customizados estão ativos no nível.

Resultado esperado

`BP_GameMode` e `BP_GameState` customizados, corretamente vinculados entre si e atribuídos ao nível de teste via Project Settings, substituindo as classes padrão da engine.

Verificando

Em Project Settings > Maps & Modes, o campo "Default GameMode" deve exibir `BP_GameMode`. Ao abrir `BP_GameMode` > Class Defaults, o campo "Game State Class" deve exibir `BP_GameState`. Um Print String temporário no Event BeginPlay do nível (chamando "Get Game State" e convertendo para `BP_GameState`) deve executar sem erro de cast, confirmando que a classe correta está ativa.

Problemas comuns

- **Esquecer de atribuir o GameMode customizado ao nível:** o projeto continua rodando com o GameMode padrão da engine sem que o estudante perceba — sempre revisar Project Settings > Maps & Modes ao final da implementação.
- **Colocar no GameMode uma variável que deveria estar no GameState (ou vice-versa):** aplicar o critério "isso precisa ser consultado por outros sistemas?" — se sim, GameState; se é regra de decisão da partida, GameMode.
- **Vincular o GameState a um GameMode diferente do atribuído ao nível:** revisar tanto o Class Defaults do GameMode quanto o Project Settings para garantir consistência.

Boas práticas

Sempre revise o Project Settings > Maps & Modes ao final de qualquer alteração no Gameplay Framework — é o ponto único de verdade sobre qual GameMode está de fato ativo no nível, independentemente do que foi criado no Content Browser.

Comparação com Unity

O GameState corresponde, em intenção, à parte do `GameManager` singleton da Unity dedicada a expor estado (por exemplo, propriedades públicas estáticas consultadas por outros scripts), ou a um `ScriptableObject` de estado compartilhado em projetos mais recentes. A diferença relevante permanece a mesma da Parte 1: a Unreal formaliza esse papel como classe nativa e replicável, enquanto a Unity resolve o mesmo problema por convenção de arquitetura da equipe. Não aprofundar mais que isso nesta semana — a comparação arquitetural mais ampla entre engines é retomada na Unidade V.

Ao final da semana

Ver seção "Ao final da semana" no Tutorial da Semana 4 — Encontro 2, que fecha o conjunto completo do Gameplay Framework desta semana (`BP_GameMode` , `BP_GameState` , `BP_PlayerController` , `BP_GameInstance`).

Desafio

Não há desafio de liberdade de solução neste encontro — a criação e atribuição de GameMode/GameState é demonstração e adaptação guiada, coerente com a transição inicial para

Checklist

- ☐ `BP_GameMode` criado em `Framework/`, herdando de Game Mode Base
- ☐ `BP_GameState` criado em `Framework/`, herdando de Game State Base
- ☐ Variável de regra adicionada ao `BP_GameMode` e variável de estado adicionada ao `BP_GameState`
- ☐ `BP_GameState` atribuído ao campo "Game State Class" do `BP_GameMode`
- ☐ `BP_GameMode` atribuído em Project Settings > Maps & Modes > Default GameMode
- ☐ Teste em modo Play confirmando que as classes customizadas estão ativas no nível

Glossário

- **GameMode:** classe nativa da Unreal responsável pela regra autoritativa da partida; existe apenas no servidor.
- **GameState:** classe nativa da Unreal responsável pelo estado compartilhado da partida, replicado para todos os clientes.
- **Gameplay Framework:** conjunto de papéis universais (GameMode, GameState, PlayerController, GameInstance, Pawn) que estruturam qualquer projeto Unreal.
- **Class Defaults:** painel do Blueprint Editor onde se configuram valores padrão e referências de classe (como o Game State Class do GameMode).

Referências

- EPIC GAMES. **Unreal Engine 5 Documentation** — Gameplay Framework in Unreal Engine. Disponível em: <https://dev.epicgames.com/documentation/en-us/unreal-engine/gameplay-framework-in-unreal-engine>.
- EPIC GAMES. **Unreal Engine Learning Library** — introdução ao Gameplay Framework (GameMode, GameState). Disponível em: <https://dev.epicgames.com/community/unreal-engine/learning>.
- UNITY TECHNOLOGIES. **Unity Manual** — GameObject e MonoBehaviour, para fins comparativos. Disponível em: <https://docs.unity3d.com/Manual/>.
- Vídeo sugerido (apoio complementar, nunca substituindo a documentação oficial): canal oficial **Unreal Engine**, vídeos introdutórios de Gameplay Framework; **Mathew Wadstein**, explicações pontuais de GameMode/GameState.